

PROYECTO FINAL - Anexos

Enric Serrano Peris

2026-06-04

El contenido de estos anexos, así como los ficheros RMarkdown de tanto este documento como el principal, se pueden consultar desde su repositorio de github.

ANEXO I. Código de geocodificación de los estancos y PVR

```
# Cargamos las siguientes librerías (asumimos que ya están instaladas):
library(sf) # Para el tratamiento de objetos espaciales
library(tidyverse) # para la limpieza, transformación y representación
library(knitr) # Para el renderizado de tablas via kable()
library(tidygeocoder) # Para la geocodificación
library(mapSpain) # Para cargar el listado de municipios de España
library(leaflet) # Para la graficación
library(RColorBrewer) # Para paletas de colores

df <- read.csv("./data/DatosEstancos.csv", skip = 2, sep = ";")

df <- df[-nrow(df),]

# Añadimos variable auxiliar con el país = "ESPAÑA"
df$Pais <- "ESPAÑA"

# Cambiamos abreviaturas por nombres completos.

abreviaturas <- c("^C\\.", "^AV\\.", "^PL\\.", "^PS\\.", "^CL\\.",
                  "^PZ\\.", "^CR\\.", "^PG\\.", "^BO\\.", "^TR\\.", "^PA\\.")

nombres <- c("C.", "AV.", "PL.", "PS.", "CL.", "PZ.", "CR.", "PG.", "BO.", "TR.", "PA.")

conteos <- sapply(abreviaturas, function(p) sum(grepl(p, df$Dirección)))
names(conteos) <- nombres

knitr::kable(data.frame(Abreviatura = names(conteos), Frecuencia = conteos), row.names = FALSE)

df$Dirección <- df$Dirección %>%
  gsub("^C\\.", "CALLE ", .) %>%
  gsub("^AV\\.", "AVENIDA ", .) %>%
  gsub("^PL\\.", "PLAZA ", .) %>%
  gsub("^PS\\.", "PASEO ", .) %>%
  gsub("^CL\\.", "CALLE ", .) %>%
  gsub("^PZ\\.", "PLAZA ", .) %>%
  gsub("^CR\\.", "CARRETERA ", .) %>%
  gsub("^PG\\.", "POLIGONO ", .) %>%
  gsub("^BO\\.", "BARRIO ", .) %>%
  gsub("^TR\\.", "TRAVESIA ", .) %>%
  gsub("^PA\\.", "PASAJE ", .) %>%
  gsub("\\.", " ", .) %>% # elimina puntos intermedios
  gsub("\\.$", " ", .) # elimina punto al final

#####

# Sacamos provincia y ccaa

provincias <- esp_get_prov() %>%
```

```

st_drop_geometry() %>%
select(cpro, ine.prov.name, nuts2.name)

df$cod_provincia <- substr(df$Estanco, 1, 2)

df <- df %>%
  left_join(provincias, by = c("cod_provincia" = "cpro"))

# Y construir la query completa:

df$query <- paste(df$Dirección, df$Municipio, df$ine.prov.name, df$nuts2.name, "España", sep = ", ")

# Comprobamos
head(df$query)

# api <- Sys.getenv("GOOGLE_API_KEY")

##### BUCLE #####

# Parámetros
tam_lote <- 500
n <- nrow(df)
lotes <- split(df, ceiling(seq_len(n) / tam_lote))

# Bucle por lotes
geo_lista <- vector("list", length(lotes))

for (i in seq_along(lotes)) {
  message("Procesando lote ", i, " de ", length(lotes), "...")

  tryCatch({
    geo_lista[[i]] <- geo(
      address = lotes[[i]]$query,
      method = "google",
      full_results = TRUE # para recuperar el campo types y demás
    )
    save(geo_lista, file = paste0("./data/geo_backup_lote_", i, ".Rdata"))
    Sys.sleep(1)

  }, error = function(e) {
    message("Error en lote ", i, ": ", e$message)
    geo_lista[[i]] <-<- NULL
  })
}

# Unimos todos los lotes en un único dataframe
df_geocodificado <- bind_rows(geo_lista)

df_geocodificado$Estanco <- df$Estanco

```

```

save(df_geocodificado, file = "./data/geocode_final.Rdata")

load("./data/geocode_final.Rdata")

#####
##### PVR #####
#####

pvr <- read.csv("./data/Listado_PVRs.csv", skip = 2, sep = ";")

pvr <- pvr[-nrow(pvr),]

names(pvr) <- c("Nombre", "Dirección", "Localidad", "Provincia")

pvr$Dirección <- pvr$Dirección %>%
  gsub("^C\\.", "CALLE ", .) %>%
  gsub("^AV\\.", "AVENIDA ", .) %>%
  gsub("^PL\\.", "PLAZA ", .) %>%
  gsub("^PS\\.", "PASEO ", .) %>%
  gsub("^CL\\.", "CALLE ", .) %>%
  gsub("^PZ\\.", "PLAZA ", .) %>%
  gsub("^CR\\.", "CARRETERA ", .) %>%
  gsub("^PG\\.", "POLIGONO ", .) %>%
  gsub("^BO\\.", "BARRIO ", .) %>%
  gsub("^TR\\.", "TRAVESIA ", .) %>%
  gsub("^PA\\.", "PASAJE ", .) %>%
  gsub("\\.", " ", .) %>% # elimina puntos intermedios
  gsub("\\.$", " ", .) # elimina punto al final

pvr$query <- paste(pvr$Dirección, pvr$Localidad, pvr$Provincia, "España", sep = ", ")

##### BUCLE #####

# Parámetros
tam_lote <- 1000
n <- nrow(pvr)
lotes <- split(pvr, ceiling(seq_len(n) / tam_lote))

# Bucle por lotes
geo_lista <- vector("list", length(lotes))

for (i in 1:length(lotes)) {
  message("Procesando lote ", i, " de ", length(lotes), "...")

  tryCatch({
    geo_lista[[i]] <- geo(
      address = lotes[[i]]$query,
      method = "arcgis",
      full_results = TRUE # para recuperar el campo types y demás
    )
  }, error = function(e) {
    message("Error al procesar lote ", i, ": ", e$message)
  })
}

```

```

    )
    save(geo_lista, file = paste0("./data/pvr_backup_lote_", i, ".Rdata"))
    Sys.sleep(1)

  }, error = function(e) {
    message("Error en lote ", i, ": ", e$message)
    geo_lista[[i]] <- NULL
  })
}

# Unimos todos los lotes en un único dataframe
pvr_geocodificado <- bind_rows(geo_lista)

# Devolvemos nombres
pvr_geocodificado$Nombre <- pvr$Nombre

save(pvr_geocodificado, file = "./data/pvr_geocode_final.Rdata")

load("./data/pvr_geocode_final.Rdata")

```

ANEXO II *Código del Proyecto final*

```
# Cargamos las siguientes librerías (asumimos que ya están instaladas):
library(sf) # Para el tratamiento de objetos espaciales
library(tidyverse) # para la limpieza, transformación y representación
library(knitr) # Para el renderizado de tablas vía kable()
library(tidygeocoder) # Para la geocodificación
library(mapspain) # Para cargar el listado de municipios de España
library(leaflet) # Para la graficación
library(RColorBrewer) # Para paletas de colores
library(patchwork) # Para unir gráficos
library(spdep) # Para el análisis Moran y LISA
library(rgeoda) # Para LISA
library(tmap) # Para graficar mapas LISA
library(RColorBrewer) # Para graficar mapas LISA
library(sfdep) # Para análisis bivariante
library(stringi) # Para quitar acentos
library(readxl) # Para leer .xlsx
```

```
# Cargamos los databases geocodificados
load("./data/geocode_final.Rdata")
load("./data/pvr_geocode_final.Rdata")
renta <- read.csv("./data/Renta_vlc.csv", sep = ";")
censos <- st_read("./data/censos_ine.geojson", quiet = T)
defunciones <- read.csv("./data/defunciones_prov_2023.csv", sep = ";")
pob <- read.csv("./data/Poblacion_provsyccaas.csv", sep = ";")
esde_tipos <- read_excel("./data/ESdE2023_MOD3_REL.xlsx",
  sheet = "Tabla 3.015",
  skip = 6)
esde_cant <- read_excel("./data/ESdE2023_MOD3_REL.xlsx",
  sheet = "Tabla 3.022",
  skip = 6)
```

```
# Cargamos mapa de España
espana <- esp_get_spain(epsg = 4326, moveCAN = T)
```

```
# Eliminamos variables innecesarias y no interesantes
```

```
df_geocodificado <- df_geocodificado %>%
  select(Estanco, lat, long, address, formatted_address, geometry.location_type, partial_match)
```

```
pvr_geocodificado <- pvr_geocodificado %>%
```

```
  select(Nombre, address, lat, long, arcgis_address, attributes.Postal, attributes.Addr_type, attributes
```

```
# Eliminamos dato chileno
```

```
chile <- which(df_geocodificado$lat < 0)
```

```
# df_geocodificado[chile,]
```

```
# Insertamos manualmente
```

```
df_geocodificado$lat[chile] <- 39.8665984964862
```

```
df_geocodificado$long[chile] <- 4.031222602250235
```

```

### Mapa estancos

# Convertimos dfs a sfs
df_geocodificado <- st_as_sf(df_geocodificado, coords = c("long","lat"), crs = 4326)

pvr_geocodificado <- st_as_sf(pvr_geocodificado, coords = c("long","lat"), crs = 4326)

# Filtramos los datos no aproximados
df_preciso <- subset(df_geocodificado, geometry.location_type == "ROOFTOP" |
                    geometry.location_type == "RANGE_INTERPOLATED")

pvr_preciso <- subset(pvr_geocodificado, attributes.StPreType != "")

# MOVEMOS DATOS CANARIOS
# Polígono de Canarias
canarias_pol <- esp_get_ccaa(ccaa = "Canarias", epsg = 4326, moveCAN = F)

# Función para mover solo puntos canarios
mover_canarias <- function(df) {
  idx <- st_intersects(df, canarias_pol, sparse = FALSE) %>% rowSums() > 0
  df_can <- df[idx, ] %>% esp_move_can()
  df_pen <- df[!idx, ]
  bind_rows(df_pen, df_can)
}

df_geocodificado <- mover_canarias(df_geocodificado)
df_preciso <- mover_canarias(df_preciso)
pvr_geocodificado <- mover_canarias(pvr_geocodificado)
pvr_preciso <- mover_canarias(pvr_preciso)

# Estancos y PVRs

graf1 <- ggplot() +
  geom_sf(data = espana) +
  geom_sf(data = pvr_geocodificado, mapping = aes(color = "PVR"), alpha = 0.2, size = 0.8) +
  geom_sf(data = df_geocodificado, mapping = aes(color = "Estancos"), alpha = 0.2, size = 0.8) +
  scale_color_manual(values = c("Estancos" = "red", "PVR" = "blue"))

# leemos geojson de secciones censales:

# Cargamos y filtramos solo Valencia ciudad
secciones_vlc <- st_read("./data/censos_ine.geojson", quiet = T) %>%
  filter(INE_MUN== "46250") %>%
  st_transform(crs = 4326)

# Comprobamos
# nrow(secciones_vlc)
# plot(st_geometry(secciones_vlc))

# Bounding box aproximado de Valencia ciudad
vlc_bbox <- st_bbox(secciones_vlc)

# Filtro rápido por bounding box (mucho más rápido que st_intersection)

```

```

estancos_vlc <- df_preciso %>%
  st_crop(vlc_bbox)

pvr_vlc <- pvr_preciso %>%
  st_crop(vlc_bbox)

# Ahora sí la intersección, pero sobre muchos menos puntos
estancos_vlc <- st_intersection(estancos_vlc, secciones_vlc)
pvr_vlc <- st_intersection(pvr_vlc, secciones_vlc)
# Contamos puntos de venta por sección censal
conteo_estancos <- estancos_vlc %>%
  st_drop_geometry() %>%
  count(ID_SECCION, name = "n_estancos")

conteo_pvr <- pvr_vlc %>%
  st_drop_geometry() %>%
  count(ID_SECCION, name = "n_pvr")

# Unimos los conteos a los polígonos de secciones censales
secciones_vlc <- secciones_vlc %>%
  left_join(conteo_estancos, by = "ID_SECCION") %>%
  left_join(conteo_pvr, by = "ID_SECCION") %>%
  mutate(
    n_estancos = replace_na(n_estancos, 0),
    n_pvr = replace_na(n_pvr, 0),
    n_total = n_estancos + n_pvr
  )

```

```

# Filtramos renta neta media por persona 2023 a nivel de sección censal

renta_vlc <- renta %>%
  filter(
    Indicadores.de.renta.media.y.mediana == "Renta neta media por persona",
    Periodo == 2023,
    !is.na(Secciones),
    str_detect(Total, "[0-9]+") # Solo procesa filas que contengan al menos un numero
  ) %>%
  mutate(
    ID_SECCION = str_extract(Secciones, "[0-9]+"),
    Total = as.numeric(str_replace_all(Total, "\\.", "")) %>%
      str_replace(",", ".")
  ) %>%
  select(ID_SECCION, renta_media = Total)

```

```

# Unimos a secciones_vlc (LA SECCION CENSAL 4625017008 NO SE ENCUENTRA EN EL DF DE RENTAS, LO ELIMINAMOS)
secciones_vlc <- secciones_vlc[secciones_vlc$ID_SECCION != 4625017008,] %>%
  left_join(renta_vlc, by = "ID_SECCION")

```

```

# Quitamos la seccion de la albufera para mejor visualizacion
secciones_vlc_red <- secciones_vlc %>%
  filter(DISTRITO != 19)

# Graficamos

```



```

p1 <- ggplot() +
  geom_sf(data = secciones_vlc_red, aes(fill = n_total), color = "white", linewidth = 0.1) +
  scale_fill_distiller(palette = "YlOrRd", direction = 1, name = "Nº puntos\nde venta") +
  theme_void()

p2 <- ggplot() +
  geom_sf(data = secciones_vlc_red, aes(fill = renta_media), color = "white", linewidth = 0.1) +
  scale_fill_distiller(palette = "YlGn", direction = 1, name = "Renta media\n(€/persona)") +
  theme_void()

graf2 <- p1 + p2

```

```

# Matriz de pesos espaciales (contigüidad queen)
vecinos <- poly2nb(secciones_vlc_red, queen = TRUE)
pesos <- nb2listw(vecinos, style = "W", zero.policy = TRUE)

# Comprobamos si hay secciones sin vecinos
# summary(vecinos)

# Estandarizar las variables
secciones_vlc_red_est <- secciones_vlc_red %>%
  mutate(
    z_puntos = scale(n_total)[,1],
    z_renta = scale(renta_media)[,1]
  )

# Moran global univariante de puntos de venta
moran_puntos <- moran.test(secciones_vlc_red_est$n_total, pesos, zero.policy = TRUE)

# Moran global univariante de renta
moran_renta <- moran.test(secciones_vlc_red_est$renta_media, pesos,
  na.action = na.omit, zero.policy = TRUE)

```

```

# Creamos funciones para mapas LISA y de significatidad
match_palette <- function(patterns, classifications, colors){
  classes_present <- base::unique(patterns)
  mat <- matrix(c(classifications,colors), ncol = 2)
  logi <- classifications %in% classes_present
  pre_col <- matrix(mat[logi], ncol = 2)
  pal <- pre_col[,2]
  return(pal)
}

lisa_map <- function(df, lisa, alpha = .05) {
  clusters <- lisa_clusters(lisa,cutoff = alpha)
  labels <- lisa_labels(lisa)
  pvalue <- lisa_pvalues(lisa)
  colors <- lisa_colors(lisa)
  lisa_patterns <- labels[clusters+1]

  pal <- match_palette(lisa_patterns,labels,colors)
  labels <- labels[labels %in% lisa_patterns]
}

```

```

df["lisa_clusters"] <- clusters
tm_shape(df) +
  tm_fill("lisa_clusters", labels = labels, palette = pal, style = "cat") +
  tm_borders(col = "gray50", lwd = 0.1)
}

significance_map <- function(df, lisa, permutations = 999, alpha = .05) {
  pvalue <- lisa_pvalues(lisa)
  target_p <- 1 / (1 + permutations)
  potential_brks <- c(.00001, .0001, .001, .01)
  brks <- potential_brks[which(potential_brks > target_p & potential_brks < alpha)]
  brks2 <- c(target_p, brks, alpha)
  labels <- c(as.character(brks2), "Not Significant")
  brks3 <- c(0, brks2, 1)

  cuts <- cut(pvalue, breaks = brks3, labels = labels)
  df["sig"] <- cuts

  pal <- rev(brewer.pal(length(labels), "Greens"))
  pal[length(pal)] <- "#D3D3D3"

  tm_shape(df) +
    tm_fill("sig", palette = pal) +
    tm_borders(col = "gray50", lwd = 0.1)
}

```

```

# Matriz de pesos espaciales con rgeoda
pesos_geoda <- queen_weights(secciones_vlc_red_est)

# LISA univariante de puntos de venta
lisa_puntos <- rgeoda::local_moran(pesos_geoda,
                                   secciones_vlc_red_est["z_puntos"])

# LISA univariante de renta
lisa_renta <- rgeoda::local_moran(pesos_geoda,
                                   secciones_vlc_red_est["z_renta"])

# LISA bivariante: puntos de venta ~ renta
lisa_biv <- local_bimoran(pesos_geoda,
                          secciones_vlc_red_est[c("z_puntos", "z_renta")])

nb <- st_contiguity(secciones_vlc_red_est, queen = T)
wt <- st_weights(nb, style = "W")

moran_bivariante <- global_moran_bv(secciones_vlc_red_est$z_puntos,
                                     secciones_vlc_red_est$z_renta,
                                     wt = wt,
                                     nb = nb)

# Calcular el p-valor bidireccional a partir del objeto de permutaciones
p_val_bi <- (sum(abs(moran_bivariante$t) >= abs(moran_bivariante$t0)) + 1) / (moran_bivariante$R + 1)

```

```

# INDICE GLOBAL BIVARIANTE
tabla1 <- kable(data.frame("Nº estancos y PVR" = c(moran_puntos$estimate[1], moran_puntos$p.value), "Re

# Mapas
lisa1 <- lisa_map(secciones_vlc_red_est, lisa_puntos)
lisa2 <- lisa_map(secciones_vlc_red_est, lisa_renta)

#lisa12 <- tmap_arrange(lisa1, lisa2, ncol = 2)
lisa1_noleg <- lisa1 + tm_layout(legend.show = FALSE, title = "Densidad de pts de \nventa de tabaco")
lisa2_noleg <- lisa2 + tm_layout(legend.show = FALSE, title = "Renta neta media")

# Extraer la leyenda de uno de ellos como elemento separado
leyenda <- lisa2 + tm_layout(legend.only = TRUE)

lisa12ex <- tmap_arrange(lisa1_noleg, lisa2_noleg, leyenda,
                        ncol = 3, widths = c(0.42, 0.42, 0.16))

lisa3 <- lisa_map(secciones_vlc_red_est, lisa_biv)

significancia1 <- significance_map(secciones_vlc_red_est, lisa_biv)

lissig <- tmap_arrange(lisa3, significancia1)

```

```

##### ANÁLISIS CCAA #####

provs <- esp_get_prov()
ccaas <- esp_get_ccaa()

# En defunciones, València no aparece formateado correctamente:
defunciones$Provincia.de.residencia[defunciones$Provincia.de.residencia == "Valencia/Val\u008ancia"] <-

# Además, para cruzar con mapSpain debemos quitar acentos a ambas tablas

defunciones <- defunciones %>%
  mutate(Provincia.de.residencia = stri_trans_general(
    Provincia.de.residencia, "Latin-ASCII"
  ))

provs <- provs %>%
  mutate(ine.prov.name = stri_trans_general(
    ine.prov.name, "Latin-ASCII"
  ))

# Filtremos

defunciones <- defunciones %>%
  filter(Causas..lista.reducida. %in% c(
    "018 Tumor maligno de la tráquea, de los bronquios y del pulmón",
    "017 Tumor maligno de la laringe",
    "064 Enfermedades crónicas de las vías respiratorias inferiores (excepto asma)"
  ))

```

```

)) %>%
  filter(Provincia.de.residencia != "Total" & Provincia.de.residencia != "Extranjero")

# Crucemos

defs <- defunciones %>%
  left_join(provs, by = c("Provincia.de.residencia" = "ine.prov.name"))

# Agreguemos defunciones a CCAA

defs <- defs %>%
  left_join(ccaas, by = "codauto") %>%
  group_by(codauto) %>%
  summarise(defunciones_tabaco = sum(Total, na.rm = TRUE))

# Convertimos a muertes por cada 100.000 habitantes, para ello preparamos pob

pob <- pob %>%
  filter(Comunidades.y.Ciudades.Autónomas != "") %>%
  filter(Sexo == "Total") %>%
  filter(Provincias == "") %>%
  filter(Periodo == 2023) %>%
  select(Comunidades.y.Ciudades.Autónomas, Total)

pob <- pob %>%
  mutate(codauto = substr(Comunidades.y.Ciudades.Autónomas, 1, 2),
         CCAA = substr(Comunidades.y.Ciudades.Autónomas, 4, NULL))

# Calculamos muertes por cada 100.000 habitantes.
defs <- defs %>%
  left_join(pob, by = "codauto")

defs$Total <- as.numeric(str_replace_all(defs$Total, "\\.", "")) %>%
  str_replace(",", ".")

defs <- defs %>%
  mutate(muertes_rel = defunciones_tabaco / Total * 100000)

# Eliminamos columnas del total en ESdE y ajustamos nombres
esde_cant <- esde_cant[3:21,-2]
esde_tipos <- esde_tipos[3:21,-2]
names(esde_cant)[1] <- "CCAA"
names(esde_tipos)[1] <- "CCAA"

# Cruzamos tablas

esde <- inner_join(esde_cant, esde_tipos, by = "CCAA")

# Estandarizamos Castilla la Mancha

esde$CCAA[esde$CCAA == "Castilla-La Mancha"] <- "Castilla - La Mancha"

```

```

defs_esde <- inner_join(defs, esde, by = "CCAA")

# Pasamos los cigarros consumidos por diarios de chr a num

defs_esde$`Entre 01 y 09 cigarrillos` <- as.numeric(defs_esde$`Entre 01 y 09 cigarrillos`)
defs_esde$`Entre 10 y 19 cigarrillos` <- as.numeric(defs_esde$`Entre 10 y 19 cigarrillos`)
defs_esde$`20 o más cigarrillos` <- as.numeric(defs_esde$`20 o más cigarrillos`)

# Calculamos la media ponderada entre prevalencia (tipo de fumador) e intensidad (cigarrillos consumidos)
defs_esde <- defs_esde %>%
  mutate(
    prevalencia = `Fumador diario` + `Fumador ocasional`,
    # Intensidad: ponderamos por peso de cada tramo de cigarrillos
    # Usamos punto medio aproximado de cada tramo: 5, 15, 25
    intensidad = (`Entre 01 y 09 cigarrillos` * 5 +
                  `Entre 10 y 19 cigarrillos` * 15 +
                  `20 o más cigarrillos` * 25) / 100,
    z_prev = scale(prevalencia)[,1],
    z_inten = scale(intensidad)[,1],
    idx_tabaquismo = (z_prev + z_inten) / 2
  )

defs_esde <- ccaas %>%
  inner_join(defs_esde, by = "codauto")

# Matriz de pesos espaciales (contigüidad queen)
vecinos2 <- poly2nb(defs_esde, queen = TRUE)
pesos2 <- nb2listw(vecinos2, style = "W", zero.policy = TRUE)

# Comprobamos si hay secciones sin vecinos
# summary(vecinos)

# Estandarizar las variables
defs_esde <- defs_esde %>%
  mutate(
    z_muertes = scale(muertes_rel)[,1]
  )

# Moran global univariante de defunciones
moran_muertes <- moran.test(defs_esde$z_muertes, pesos2, zero.policy = TRUE)

# Moran global univariante de tabaquismo
moran_tab <- moran.test(defs_esde$idx_tabaquismo, pesos2,
                        na.action = na.omit, zero.policy = TRUE)

# Matriz de pesos espaciales con rgeoda
pesos_geoda2 <- queen_weights(defs_esde)

```

```

# LISA univariante de muertes por cada 100.000 habitantes
lisa_muertes <- rgeoda::local_moran(pesos_geoda2,
                                   defs_esde["muertes_rel"])

# LISA univariante de tabaquismo
lisa_tab <- rgeoda::local_moran(pesos_geoda2,
                               defs_esde["idx_tabaquismo"])

# LISA bivariante: muertes ~ tabaquismo
lisa_biv2 <- local_bimoran(pesos_geoda2,
                          defs_esde[c("muertes_rel", "idx_tabaquismo")])

nb2 <- st_contiguity(defs_esde, queen = T)

tiene_vecinos <- which(sapply(nb2, length) > 1)
defs_esde_cont <- defs_esde[tiene_vecinos, ]

nb2 <- st_contiguity(defs_esde_cont, queen = T)

wt2 <- st_weights(nb2, style = "W", allow_zero = T)

moran_bivariante2 <- global_moran_bv(defs_esde_cont$muertes_rel,
                                     defs_esde_cont$idx_tabaquismo,
                                     wt = wt2,
                                     nb = nb2)

# Calcular el p-valor bidireccional a partir del objeto de permutaciones
p_val_bi2 <- (sum(abs(moran_bivariante2$t) >= abs(moran_bivariante2$t0)) + 1) / (moran_bivariante2$R + 1)

# INDICE GLOBAL BIVARIANTE
tabla2 <- kable(data.frame("Idx.defunciones.pulmonares" = c(moran_muertes$estimate[1], moran_muertes$p.value)))

# Mapas
lisa3 <- lisa_map(defs_esde, lisa_muertes)
lisa4 <- lisa_map(defs_esde, lisa_tab)

#lisa34 <- tmap_arrange(lisa3, lisa4, ncol = 2)
lisa3_noleg <- lisa3 + tm_layout(legend.show = FALSE, title = "Muertes pulmonares \n/ 10000 hab.")
lisa4_noleg <- lisa4 + tm_layout(legend.show = FALSE, title = "Índice tabaquismo")

lisa34ex <- tmap_arrange(lisa3_noleg, lisa4_noleg, leyenda,
                        ncol = 3, widths = c(0.42, 0.42, 0.16))

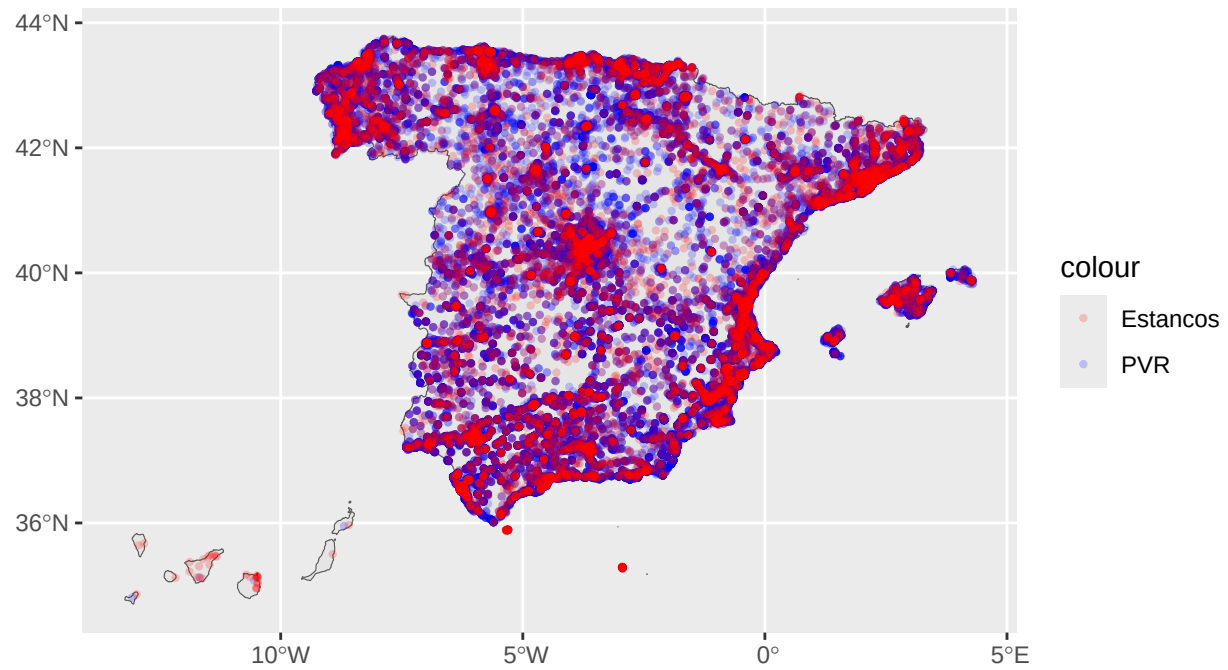
lisa5 <- lisa_map(defs_esde, lisa_biv2)

significancia2 <- significance_map(defs_esde, lisa_biv2)

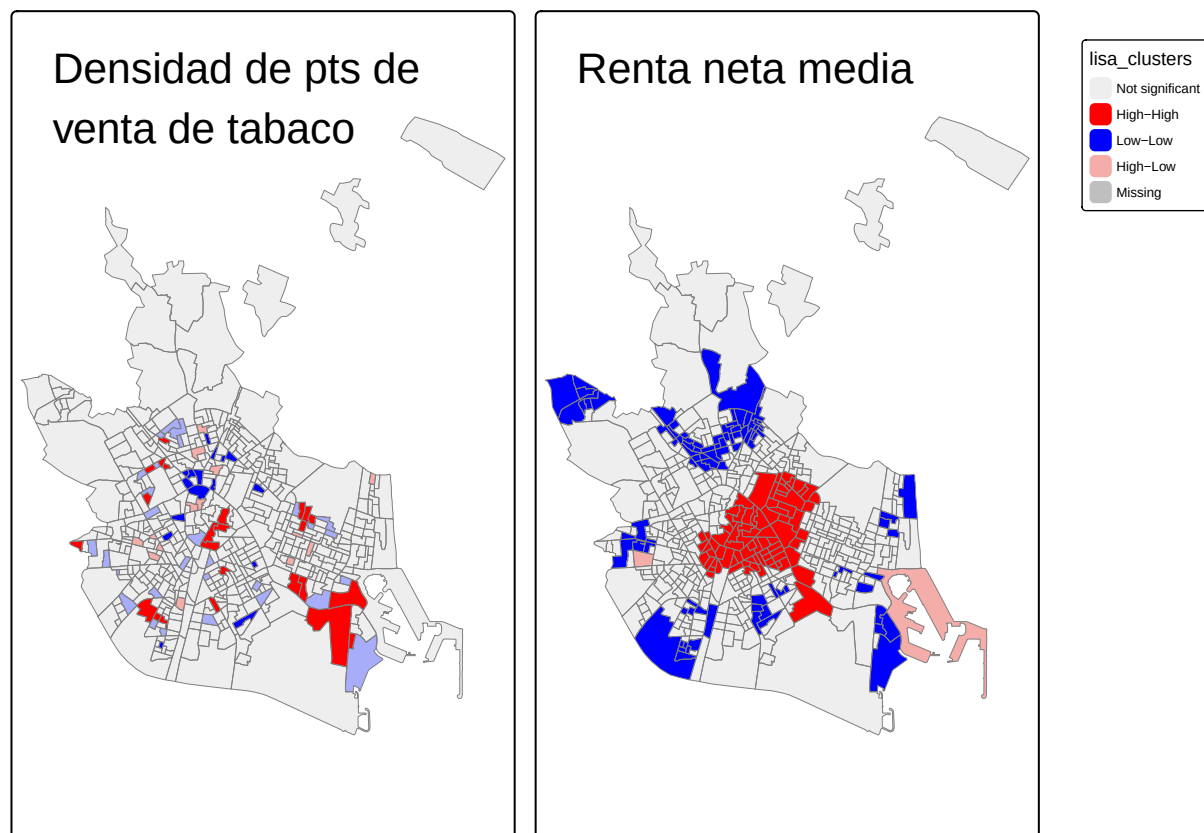
lissig2 <- tmap_arrange(lisa5, significancia2)

```

Anexo III *Distribución de estancos y PVR en España.*



Anexo IV. Mapas LISAs univariantes del nº de puntos de venta de tabaco (izquierda) y renta por sección censal valenciana (derecha), excepto el distrito 19.



Anexo V. Mapas LISAs univariantes del nº de defunciones por enfermedades pulmonares y tabaquismo en España

